



Lab Report – EE-170L Computer Programming

NAME	Danyal Hassan
ID	750
SEMESTER	2nd
LAB	Programming
DATE	26/06/2026

COMPUTER PROGRAMMING

Electrical Engineering Department — UET Nowshera

LAB 5

Lab Title: The while Loop in C++

1. OBJECTIVE

The objectives of this lab are:

- To gain a solid understanding of the concept and syntax of the while loop in C++.
- To examine how a nested while loop functions and when it is used.
- To understand the do...while loop and how its behaviour differs from the standard while loop.
- To develop programs that repeat a set of instructions based on a given condition.
- To apply loop structures to practical problems, including printing number sequences and computing cumulative sums.

2. THEORY

The while Loop

The while loop is a control flow construct that continues to execute a block of code as long as a specified condition evaluates to true. Since the condition is checked at the beginning of every iteration, the loop body will not run at all if the condition is false initially.

Syntax:

```
while (condition)
{
    // statements to repeat
}
```

Working of the while loop:

- The condition is evaluated.
- If true, the loop body executes.
- After each execution, the condition is re-evaluated.
- The loop continues until the condition becomes false.

Nested while Loop

A nested while loop refers to a while loop that is contained within another while loop. The outer loop governs how many times the inner loop is executed. For each single pass of the outer loop, the inner loop runs through all of its iterations from start to finish. This structure is frequently applied when working with tables, geometric patterns, and matrix-based computations.

Syntax concept:

```
while(condition1)           // outer loop
{
    while(condition2)       // inner loop
    {
        // inner loop statements
    } }
```

The do...while Loop

The do...while loop closely resembles the while loop but carries one key distinction: the body of the loop is guaranteed to execute at least once, since the condition is evaluated after the body has already run rather than before.

Syntax:

```
do {
    // statements
}
while(condition); // condition checked after each iteration
```

3. SOFTWARE USED

- DEV-C++ IDE
- C++ Compiler (bundled with DEV-C++)

4. TASKS

Task 1 — Display Numbers Using while Loop

A C++ program was implemented using a while loop to print the numbers 1 through 10. The loop variable is initialized to 1 and increases by one with each iteration, and the loop terminates once the counter goes beyond 10.

```
#include <iostream> using
namespace std;

int main()
{
    int i = 1; // initialise counter

    while(i <= 10) // loop runs while i is 10 or less
    {
        cout << i << endl; // print current number
        i++;                // increment counter    }

    return 0;
}
```

Task 2 — Display Numbers Using do...while Loop

The identical output — numbers 1 to 10 — was generated using a do...while loop. Since the condition is evaluated only after the loop body executes, the body is guaranteed to run at least one time, regardless of the counter's starting value.

```
#include <iostream> using
namespace std;

int main()
{
    int i = 1; // initialise counter

do
{
    cout << i << endl; // body executes before condition is checked
    i++;
} while(i <= 10); // condition checked after each iteration

    return 0;
}
```

Task 3 — Sum of Numbers Using while Loop

This program prompts the user to provide a positive integer and then employs a while loop to compute the cumulative sum of all integers from 1 up to and including the number entered.

```
#include <iostream> using
namespace std;

int main()
{
    int num, i = 1, sum = 0;

    cout << "Enter a positive integer: ";
    cin >> num;

    while(i <= num) // add each number from 1 to num
    {
        sum += i; // accumulate the sum
        i++;
    }

    cout << "Sum = " << sum << endl;

    return 0;
}
```

5. CONCLUSION

This lab covered the while loop and the do...while loop in C++, with both being implemented through hands-on programs. The fundamental distinction between the two — namely, when the condition is evaluated relative to the loop body — was highlighted through working examples. Loop constructs were used to generate number sequences and to compute running totals, demonstrating their central role in any program that involves repetition or iteration.